



* 1. What is OOPs ?

- **Definition:** Programming paradigm based on 'Objects' containing data and logic.
- **Core Goal:** Make code modular, reusable, scalable, and easy to maintain.
- **Python Support:** Python is dynamically typed and fully supports multi-paradigm OOP.

* 4 Main Pillars of OOPs:

1. **Encapsulation** - Data hiding & wrapping
2. **Inheritance** - Reusing parent attributes
3. **Polymorphism** - Many forms, same interface
4. **Abstraction** - Hiding internal complexity

CLASS (Blueprint)

- Attributes: brand, hp
- Methods: drive(), brak
- Template in Memory

Instantiates →

OBJECT 1: car1 = Car('Tesla')

- RAM Address: 0x7fa920

OBJECT 2: car2 = Car('BMW')

- RAM Address: 0x7fa948

* Python Class & Object Syntax:

```
class Car:
```

```
    def __init__(self, brand, hp):
```

```
        self.brand = brand    # Instance attribute
```

```
        self.hp = hp
```

```
my_car = Car('Tesla', 450)
```

```
# Object instantiation
```

```
print(my_car.brand)
```

```
# Output: Tesla
```



* 2. Understanding `__init__` and `self` :

- `__init__()`: Special dunder method invoked automatically when a new object is created.
- `self`: Reference to the current instance of the class; binds attributes to the instance.
- **Instance Attributes**: Unique to each instance; defined inside `__init__` with `self`.
- **Class Attributes**: Shared across all instances; defined directly inside the class body.

* Instance vs Class Attributes Example:

```
class Student:
    school_name = 'Tech High'    # Class attribute (shared)

    def __init__(self, name, roll):
        self.name = name         # Instance attribute
        self.roll = roll

s1 = Student('Alice', 101)
```

Attribute Type	Scope	Access Syntax
Instance Var	Specific to single object	<code>self.var_name</code>
Class Var	Shared across all objects	<code>ClassName.var_name</code>
Local Var	Only inside a single method	<code>var_name</code> inside <code>def</code>



* 3. Encapsulation & Access Modifiers :

- **Definition:** Restricting direct access to an object's internal state to prevent tampering.
- **Public Members:** Accessible anywhere inside or outside the class (e.g. `self.name`).
- **Protected Members:** Prefix with single underscore '`_`'; convention for internal use.
- **Private Members:** Prefix with double underscore '`__`'; triggers Python name mangling.

* Python Access Modifier Rules:

Public - `self.name` (Anywhere)

Protected - `self._balance` (Internal)

Private - `self.__pin` (Mangled)

Properties - `@property` decorator

* Getter and Setter Implementation:

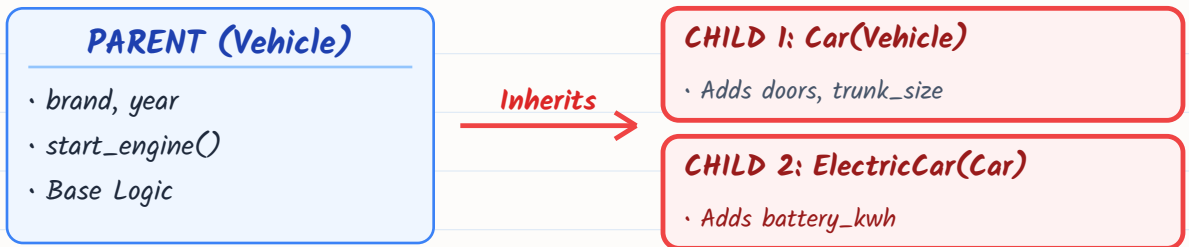
```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance        # Private variable

    @property
    def balance(self):
        return self.__balance          # Getter method
```



* 4. Inheritance & Method Overriding :

- **Definition:** Mechanism where a child class derives attributes and methods from a parent class.
- **Code Reusability:** Avoid DRY (Don't Repeat Yourself) violations across large codebases.
- **super():** Built-in function used to call the parent class methods or `__init__` constructor.
- **Method Overriding:** Child class defines a method with the same name as the parent.



* Inheritance with super() Syntax:

```
class Animal:
    def __init__(self, name): self.name = name
    def speak(self): return 'Sound'

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)      # Call parent cons
        self.breed = breed
```



* 5. Magic (Dunder) Methods Cheat Sheet :

- **Dunder Methods:** Double Underscore methods like `__init__`, `__str__`, and `__repr__`.
- **Operator Overloading:** Allows custom classes to respond to `+`, `-`, `*`, `len()`, and `print()`.
- **Pythonic Code:** Enables objects to behave seamlessly with Python's built-in ecosystem.

Dunder Method	Triggered By	Example Use
<code>__str__(self)</code>	<code>str(obj)</code> , <code>print(obj)</code>	User-friendly string
<code>__repr__(self)</code>	<code>repr(obj)</code> , terminal	Unambiguous debug ou
<code>__len__(self)</code>	<code>len(obj)</code>	Return collection si
<code>__eq__(self, o)</code>	<code>obj1 == obj2</code>	Compare equality
<code>__add__(self, o)</code>	<code>obj1 + obj2</code>	Overload + operator

* Custom Dunder Methods in Action:

```
class Book:
    def __init__(self, title, pages):
        self.title, self.pages = title, pages
    def __str__(self): return f'{self.title} ({self.page
    def __len__(self): return self.pages

b = Book('Clean Code', 464)
print(b)          # Output: Clean Code (464p)
```